



UNIVERSITÀ DI BOLOGNA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA E SCIENZE
INFORMATICHE

TrailSafe

Componenti del gruppo:

Sajmir Buzi
Margherita Balzoni

Anno accademico 2025–2026

Indice

1	Introduzione	3
2	Requisiti	4
2.1	Requisiti utente	4
2.2	Requisiti funzionali	4
2.3	Requisiti non funzionali	5
3	Design e Architettura	6
3.1	Design del sistema	6
3.2	Architettura generale del sistema	6
3.3	Architettura Backend	6
3.4	Architettura Frontend	7
3.5	Sicurezza	7
3.6	Architettura delle interfacce utente – Mockup	8
3.7	Design di dettaglio – Diagrammi UML	12
3.8	Target User Analysis	14
3.8.1	Persona: Marco	14
3.8.2	Persona: Laura	14
3.8.3	Persona: Paolo	15
3.9	Prototipo Finale	16
4	Tecnologie utilizzate	20
4.1	Stack tecnologico MEVN	20
4.2	Tecnologie Backend	20
4.3	Tecnologie Frontend	20
4.4	Servizi di mappe e informazioni ambientali	21
4.5	Integrazione del servizio di Intelligenza Artificiale	21
5	Codice	21
5.1	Recupero dei percorsi più popolari	21
5.2	Creazione di una segnalazione	22
5.3	Middleware di autenticazione	22
5.4	Registrazione di un nuovo utente	23
5.5	Autenticazione utente	23
5.6	Recupero delle informazioni meteo	24
5.7	Ricerca intelligente dei percorsi	24
5.8	Filtraggio dei percorsi	25
5.9	Visualizzazione dei percorsi su mappa	26
6	Testing	27
6.1	Strategia di testing	27
6.2	Testing del Backend	27
6.3	Testing del Frontend e delle pagine	27
6.4	Testing delle funzionalità principali	28
7	Deployment	29
7.1	Setup iniziale e scaffolding	29
7.2	Avvio dei servizi in ambiente di sviluppo	29
7.3	Comandi di avvio	29
7.4	Configurazione del proxy	29
7.5	Gestione dell'utente amministratore e cliente	30

1 Introduzione

Negli ultimi anni, l'interesse verso le attività outdoor, in particolare l'escursionismo, ha registrato una crescita significativa. Tuttavia, l'aumento del numero di utenti sui sentieri comporta anche un incremento dei rischi legati a condizioni ambientali variabili, scarsa informazione sullo stato dei percorsi e difficoltà nella gestione delle emergenze. In questo contesto emerge la necessità di strumenti digitali in grado di supportare gli escursionisti nella pianificazione e nello svolgimento delle attività in modo più sicuro e consapevole.

TrailSafe è una Web App interattiva progettata con l'obiettivo di migliorare la sicurezza degli escursionisti attraverso la fornitura di informazioni aggiornate in tempo reale sui sentieri, sulle condizioni ambientali e sui potenziali rischi presenti lungo i percorsi. La piattaforma consente agli utenti di consultare mappe interattive, ricevere notifiche dinamiche e accedere a dati utili per valutare la difficoltà e la percorribilità dei sentieri.

Il sistema si rivolge sia agli escursionisti, offrendo strumenti di supporto alla pianificazione e alla sicurezza durante le escursioni, sia agli amministratori, che possono gestire in modo centralizzato i contenuti della piattaforma, aggiornare lo stato dei percorsi e moderare le segnalazioni inviate dagli utenti.

L'applicazione è sviluppata come sistema web moderno e modulare, basato su tecnologie JavaScript full-stack, con una netta separazione tra frontend e backend. Questa scelta progettuale consente di garantire scalabilità, manutenibilità e possibilità di estensione futura del sistema, rendendo TrailSafe una soluzione flessibile e adatta a diversi contesti territoriali.

L'obiettivo finale del progetto è quello di contribuire alla riduzione dei rischi associati alle attività escursionistiche, migliorando la consapevolezza degli utenti e favorendo un utilizzo più sicuro e responsabile dei sentieri naturali.

2 Requisiti

In questa sezione vengono descritti i requisiti del sistema TrailSafe, suddivisi in requisiti utente, requisiti funzionali e requisiti non funzionali. Tale suddivisione consente di analizzare in modo strutturato le esigenze degli utenti finali e di definire con chiarezza le funzionalità e le caratteristiche qualitative che il sistema deve garantire.

2.1 Requisiti utente

L'utente finale del sistema TrailSafe è rappresentato principalmente da escursionisti con diversi livelli di esperienza, interessati a pianificare e svolgere attività outdoor in modo sicuro e consapevole. A questi si affianca la figura dell'amministratore, incaricato della gestione dei contenuti e del monitoraggio dello stato dei percorsi.

Gli utenti escursionisti si aspettano dal sistema di:

- Consultare una mappa interattiva dei sentieri disponibili;
- Visualizzare informazioni aggiornate sulle condizioni ambientali e meteorologiche;
- Valutare la difficoltà dei percorsi in base a dati reali e dinamici;
- Ricevere notifiche e avvisi relativi a rischi, chiusure o cambiamenti dei percorsi;
- Salvare i sentieri di interesse per una consultazione futura;
- Inviare segnalazioni riguardanti anomalie o situazioni di pericolo riscontrate sul territorio.

Gli amministratori del sistema si aspettano di poter:

- Gestire l'elenco dei sentieri disponibili;
- Aggiornare lo stato e le caratteristiche dei percorsi;
- Moderare e validare le segnalazioni inviate dagli utenti;
- Pubblicare avvisi e comunicazioni rilevanti per la sicurezza;
- Monitorare l'attività generale della piattaforma.

2.2 Requisiti funzionali

Il sistema TrailSafe deve fornire le seguenti funzionalità principali:

- Registrazione e autenticazione degli utenti;
- Accesso differenziato alle funzionalità in base al ruolo dell'utente (escursionista o amministratore);
- Visualizzazione dei sentieri tramite mappa interattiva;
- Visualizzazione di punti di interesse e aree critiche lungo i percorsi;
- Aggiornamento automatico delle informazioni meteo e ambientali;
- Calcolo dinamico della difficoltà dei sentieri;
- Gestione delle notifiche e degli avvisi in tempo reale;
- Inserimento e gestione delle segnalazioni da parte degli utenti;
- Creazione, modifica e rimozione dei sentieri da parte degli amministratori.

2.3 Requisiti non funzionali

Oltre ai requisiti funzionali, il sistema deve soddisfare una serie di requisiti non funzionali fondamentali per garantire qualità e affidabilità:

- Usabilità: l'interfaccia deve essere intuitiva e facilmente utilizzabile anche da utenti con scarsa esperienza tecnologica;
- Prestazioni: il sistema deve garantire tempi di risposta adeguati anche in presenza di numerosi utenti simultanei;
- Affidabilità: le informazioni fornite devono essere accurate e aggiornate;
- Sicurezza: i dati degli utenti devono essere protetti mediante meccanismi di autenticazione e autorizzazione;
- Scalabilità: l'architettura deve consentire l'estensione del sistema a nuovi servizi e funzionalità;
- Portabilità: l'applicazione deve essere accessibile tramite browser su diversi dispositivi e piattaforme.

3 Design e Architettura

In questa sezione vengono descritte le scelte progettuali adottate per la realizzazione di TrailSafe, analizzando il modello di design seguito e l'architettura generale del sistema.

L'obiettivo è illustrare come le decisioni progettuali abbiano contribuito a garantire scalabilità, manutenibilità e affidabilità dell'applicazione.

3.1 Design del sistema

Il processo di sviluppo di TrailSafe ha seguito un approccio incrementale e iterativo, ispirato ai principi delle metodologie Agile. Le funzionalità del sistema sono state sviluppate progressivamente, dando priorità agli aspetti fondamentali legati alla sicurezza e alla fruibilità dell'interfaccia.

Particolare attenzione è stata posta sull'esperienza utente, adottando principi di User-Centered Design, tale è una filosofia di progettazione e un processo nel quale ai bisogni, ai desideri e ai limiti dell'utente sul prodotto finale è data grande attenzione in ogni passo del processo di progettazione per massimizzare l'usabilità del prodotto stesso.

L'interfaccia è stata progettata per essere chiara e intuitiva, riducendo al minimo la complessità operativa e consentendo un accesso immediato alle informazioni rilevanti. Durante lo sviluppo sono stati seguiti i principi di progettazione KISS (Keep It Simple, Stupid) e DRY (Don't Repeat Yourself), al fine di mantenere il codice semplice, leggibile e facilmente manutenibile come imparato in altri corsi.

L'applicazione è stata progettata secondo i principi del Responsive Design, garantendo una corretta visualizzazione e un utilizzo efficace su dispositivi con differenti dimensioni e risoluzioni.

3.2 Architettura generale del sistema

TrailSafe è basata su un'architettura client-server, con una chiara separazione tra la parte di frontend e il backend. Tale separazione consente di isolare le responsabilità dei diversi componenti e di facilitare l'evoluzione futura del sistema.

Il backend espone un insieme di API RESTful che permettono al client di accedere alle funzionalità del sistema. Il frontend comunica con il backend tramite richieste HTTP, occupandosi esclusivamente della presentazione dei dati e dell'interazione con l'utente.

L'architettura del sistema consente inoltre l'integrazione di servizi esterni, come servizi di aggiornamento meteo, servizi per integrazione di intelligenza artificiale per informazioni delle escursioni tramite Gemini, e sistemi di notifica fondamentali per garantire la dinamicità e l'aggiornamento in tempo reale delle informazioni.

3.3 Architettura Backend

Il backend dell'applicazione è sviluppato utilizzando Node.js ed Express.js ed è strutturato in modo modulare per favorire la manutenibilità del codice. L'architettura aderisce al modello REST, identificando le principali entità del dominio applicativo e fornendo operazioni CRUD, ovvero operazioni di Create, Read, Update e Delete per ciascuna di esse.

Le principali entità gestite dal sistema includono:

- Utente;
- Sentiero;
- Segnalazione;

- Notifica;
- Informazioni ambientali.
- Informazioni Escursioni tramite AI

Il backend è organizzato in moduli distinti che comprendono:

- **Routes**, responsabili della definizione degli endpoint REST;
- **Controllers**, incaricati della gestione della logica applicativa;
- **Models**, che rappresentano la struttura dei dati persistenti;
- **Services**, utilizzati per l'integrazione di servizi esterni e per la gestione di funzionalità trasversali come notifiche e aggiornamenti meteo;
- **Middleware**, dedicati alla gestione dell'autenticazione e dell'autorizzazione degli utenti.

I dati vengono memorizzati all'interno di un database NoSQL nel nostro caso per usare lo stile MEVN e stato usato MongoDB, scelto per la sua flessibilità e per la capacità di adattarsi a strutture dati eterogenee e in continua evoluzione.

3.4 Architettura Frontend

Il frontend di TrailSafe è sviluppato utilizzando il framework Vue.js, adottando una struttura basata su componenti. Questa scelta consente di suddividere l'interfaccia in varie unità riutilizzabili e indipendenti, migliorando in maniera ampia l'organizzazione del codice e la manutenibilità temporale del progetto.

L'applicazione frontend è organizzata in:

- **Pages**, che rappresentano le principali viste dell'applicazione;
- **Components**, utilizzati per elementi riutilizzabili dell'interfaccia;
- **Services**, che gestiscono la comunicazione con il backend e con i servizi esterni;
- **Assets e fogli di stile**, dedicati alla gestione dell'aspetto grafico.

A differenza del frontend si occupa della gestione dello stato dell'applicazione e della visualizzazione dinamica dei dati, aggiornando l'interfaccia in base alle informazioni ricevute dal backend e alle interazioni dell'utente.

3.5 Sicurezza

La sicurezza rappresenta un aspetto centrale del sistema TrailSafe, in quanto vengono gestiti alcuni dati personali degli utenti e informazioni sensibili relative ai percorsi. Per questo motivo, l'accesso alle funzionalità del sistema è regolato da meccanismi di autenticazione e autorizzazione.

L'autenticazione degli utenti avviene mediante l'utilizzo di token, che permettono di verificare l'identità dell'utente ad ogni richiesta al backend. Le operazioni sensibili sono protette e accessibili esclusivamente agli utenti autorizzati, in particolare agli amministratori del sistema.

Queste scelte progettuali consentono di ridurre i rischi legati all'accesso non autorizzato e di garantire un livello di sicurezza adeguato per un'applicazione web come nel nostro caso.

3.6 Architettura delle interfacce utente – Mockup

Durante la fase di progettazione del sistema TrailSafe sono stati realizzati dei mockup per versione web delle principali interfacce utente, con l'obiettivo di definire in modo chiaro la struttura dell'applicazione e il flusso di navigazione prima della fase di implementazione.

I mockup rappresentano una prima versione concettuale dell'interfaccia e consentono di visualizzare la disposizione degli elementi principali, facilitando l'individuazione delle funzionalità essenziali e migliorando l'esperienza utente. In particolare, sono state progettate le seguenti interfacce:

- Pagina iniziale dell'applicazione;
- Interfaccia di login e registrazione;
- Mappa interattiva dei sentieri;
- Pagina del singolo sentiero con le varie informazioni
- Pagina generativa dei percorsi tramite AI generativa
- Area utente per la gestione dei percorsi salvati;
- Area amministratore per la gestione dei sentieri e delle segnalazioni.

Le figure seguenti mostrano alcuni mockup significativi dell'applicazione TrailSafe, che illustrano la struttura generale delle pagine e l'organizzazione delle funzionalità principali.

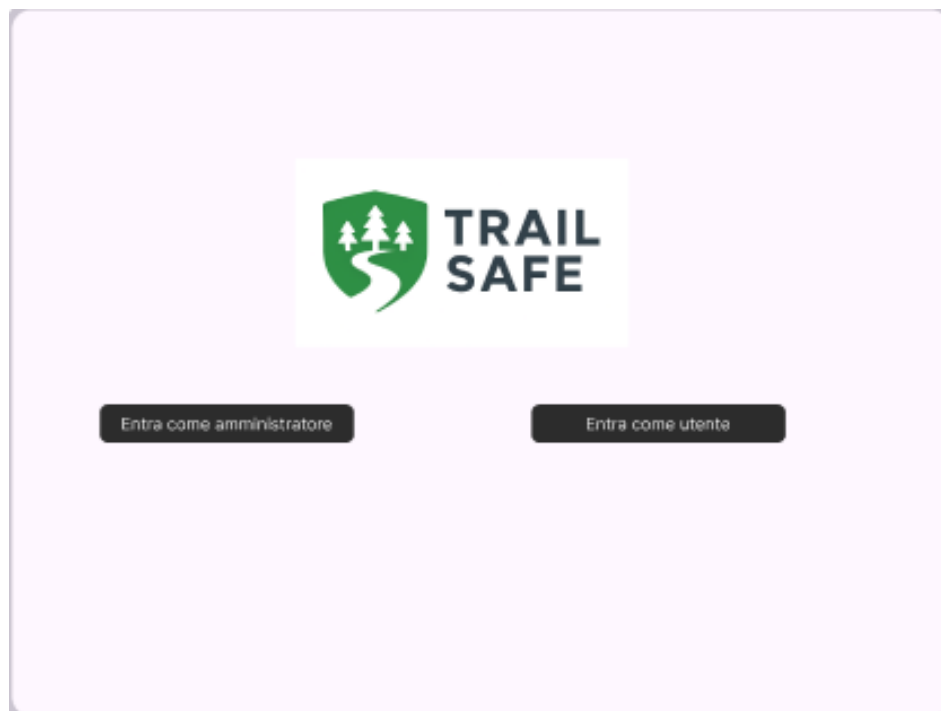


Figura 1: mockup login



Figura 2: mockup home

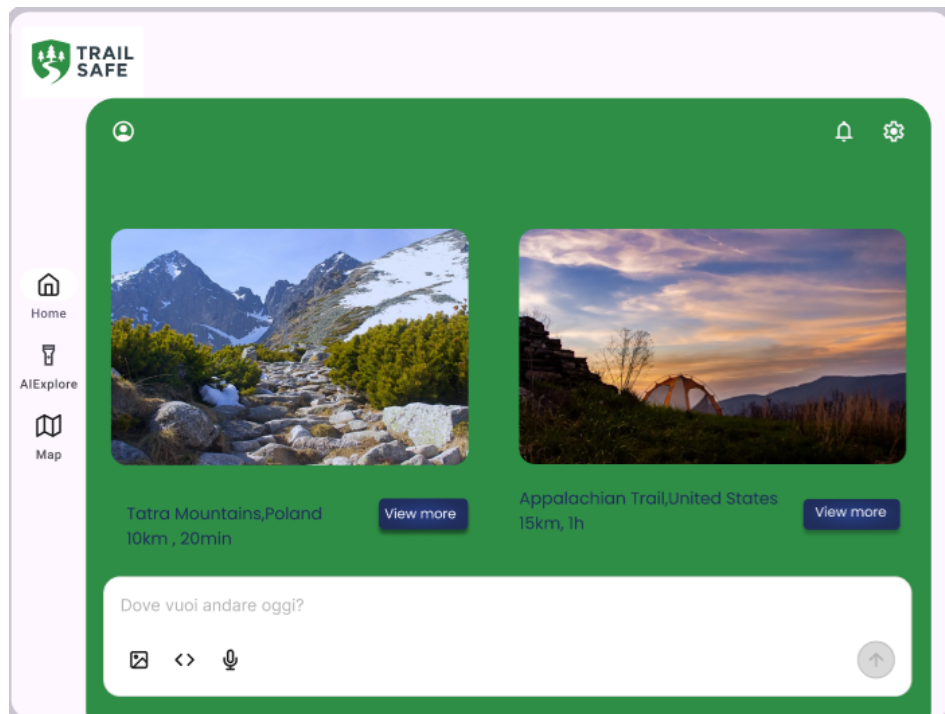


Figura 3: mockup AI

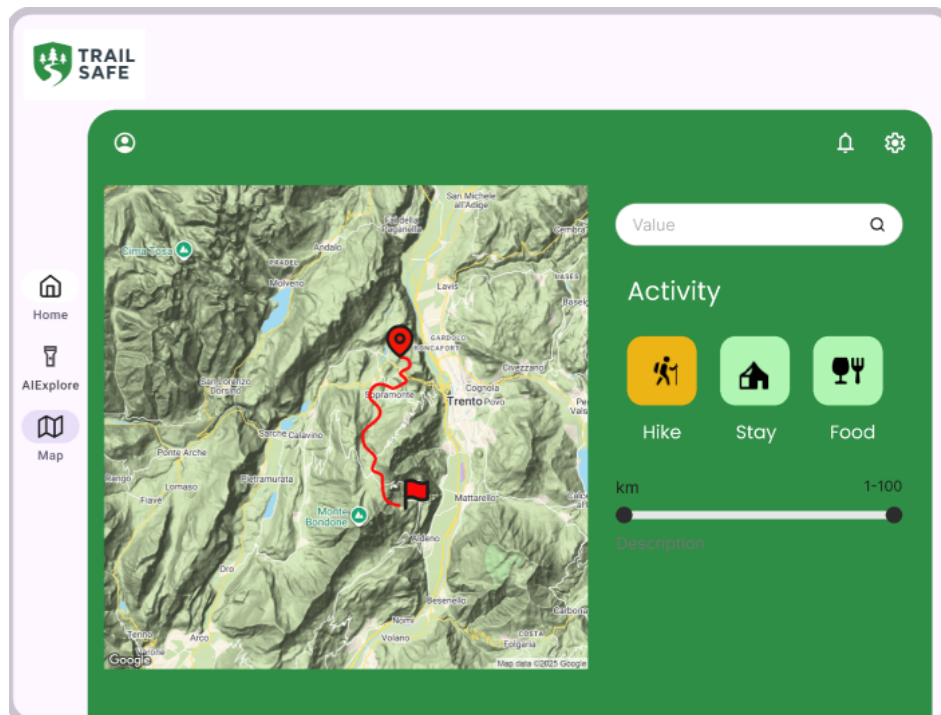


Figura 4: mockup map

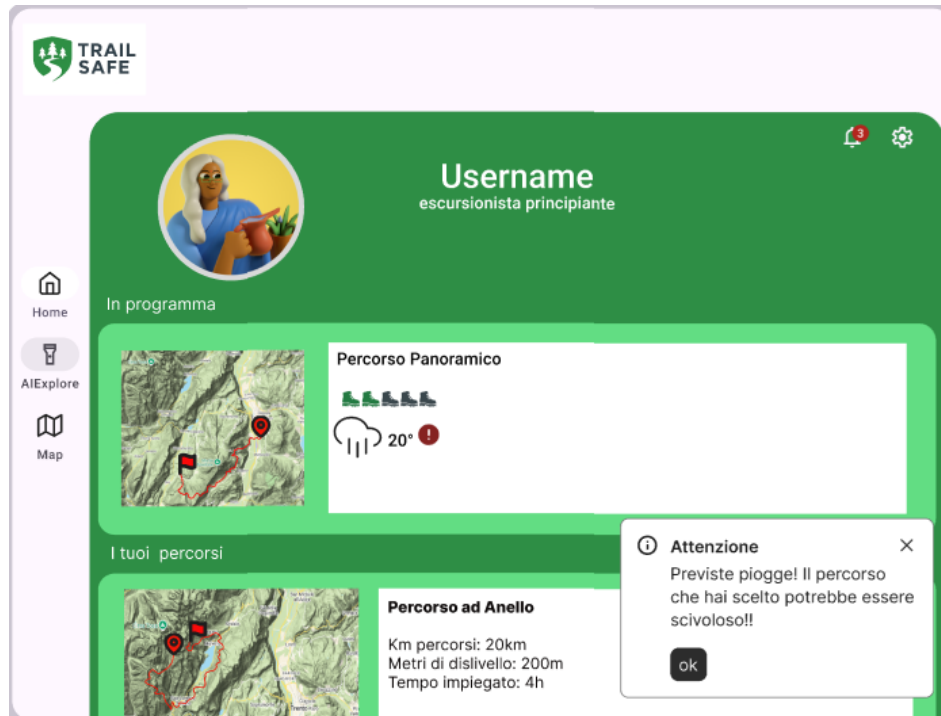


Figura 5: mockup profile



Figura 6: mockup singolo percorso info

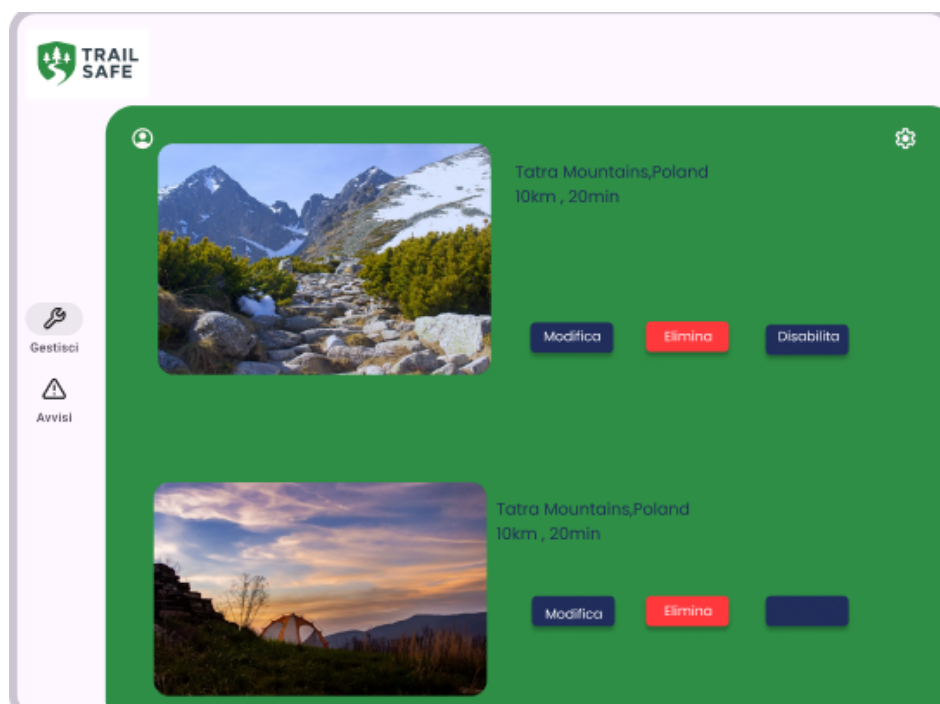


Figura 7: mockup gestione parte admin

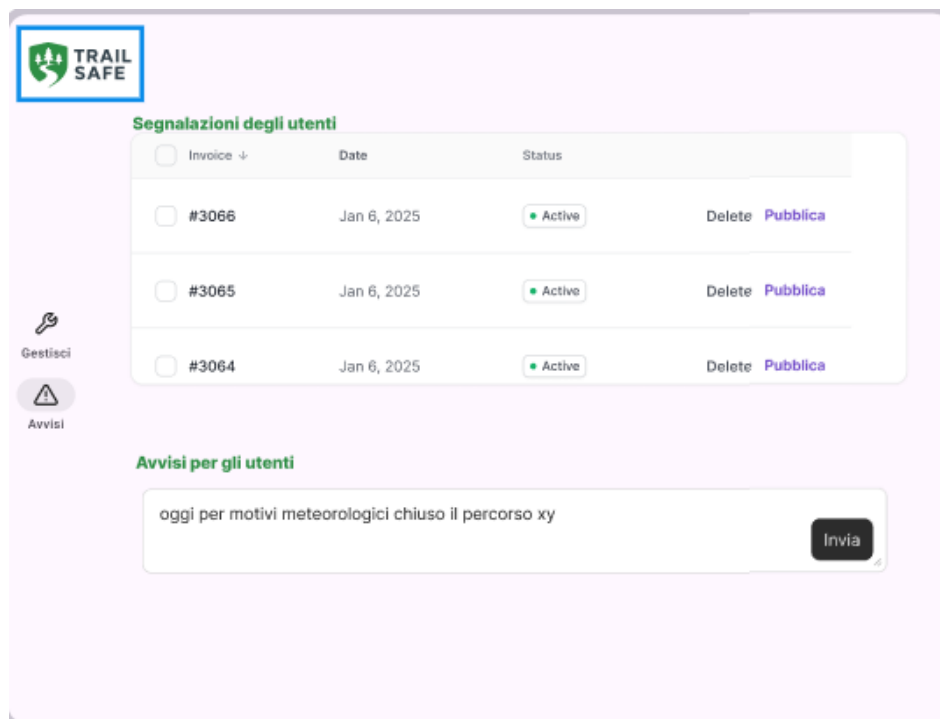


Figura 8: mockup segnalazioni parte admin

3.7 Design di dettaglio – Diagrammi UML

Per descrivere in modo formale la struttura interna del sistema TrailSafe sono stati utilizzati diagrammi UML (Unified Modeling Language). Tali diagrammi permettono di rappresentare le entità principali del dominio applicativo e le relazioni esistenti tra di esse, facilitando la comprensione dell'architettura del sistema.

In particolare, il diagramma delle classi rappresenta le principali entità gestite dal backend dell'applicazione, tra cui:

- Utente;
- Sentiero;
- Segnalazione;
- Notifica;
- Dati ambientali.

Ogni entità è caratterizzata da un insieme di attributi e metodi che definiscono il comportamento e le operazioni consentite. Le relazioni tra le classi descrivono i legami logici tra gli elementi del sistema, come l'associazione tra utenti e segnalazioni o tra sentieri e notifiche.

Il diagramma UML consente inoltre di evidenziare la separazione delle responsabilità tra i diversi componenti del sistema, supportando un'architettura modulare e facilmente estendibile.

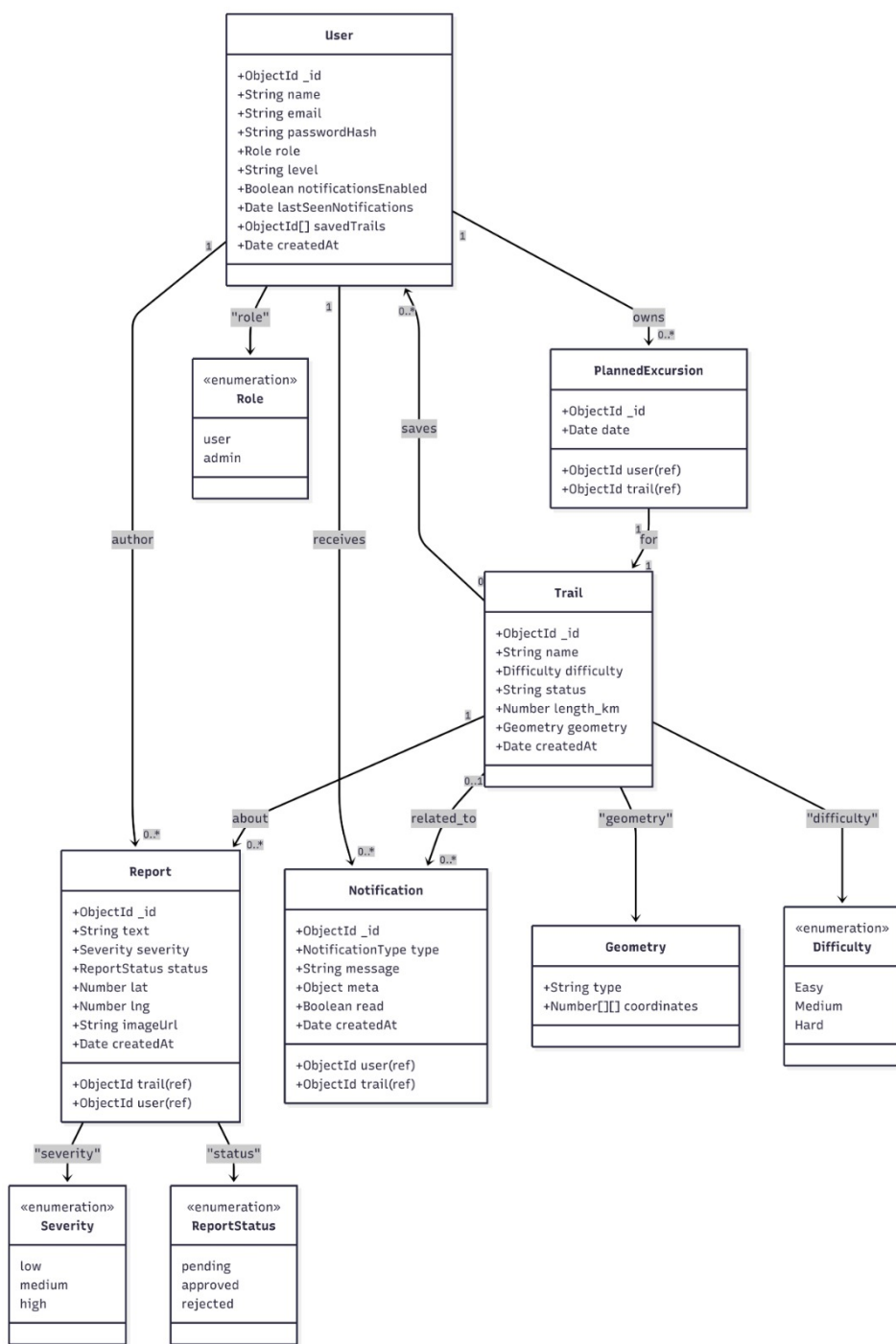


Figura 9: Diagramma UML delle principali entità del sistema TrailSafe

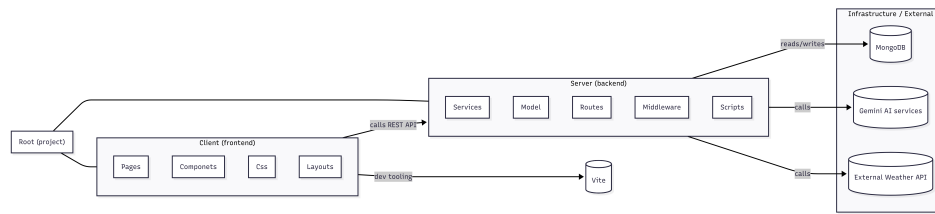


Figura 10: Diagramma UML dei package del sistema TrailSafe

3.8 Target User Analysis

Il sistema TrailSafe è rivolto a diverse tipologie di utenti che praticano attività escursionistiche con differenti livelli di esperienza, oltre alla figura dell'amministratore incaricato della gestione della piattaforma. L'analisi del target utente consente di individuare i principali profili che interagiscono con il sistema e di comprendere le loro esigenze, al fine di progettare un servizio efficace e reattivo.

3.8.1 Persona: Marco

Marco è uno studente universitario di 22 anni che ha recentemente iniziato ad avvicinarsi all'escursionismo. Possiede un livello di esperienza base e predilige sentieri semplici e ben segnalati. Utilizza frequentemente applicazioni digitali per pianificare le proprie attività all'aperto.

Scenario d'uso: Marco desidera organizzare un'escursione in sicurezza.

- Marco accede alla piattaforma TrailSafe;
- Consulta la mappa interattiva dei sentieri;
- Filtra i percorsi in base al livello di difficoltà;
- Visualizza le condizioni meteo e la percorribilità del sentiero;
- Salva il percorso di interesse tra i preferiti;
- Riceve notifiche in caso di variazioni delle condizioni ambientali.

3.8.2 Persona: Laura

Laura è una professionista di 35 anni con una buona esperienza escursionistica. Ama affrontare percorsi di media e alta difficoltà e presta particolare attenzione alle condizioni ambientali e ai potenziali rischi lungo i sentieri.

Scenario d'uso: Laura pianifica un'escursione impegnativa.

- Laura accede alla piattaforma TrailSafe;
- Analizza sentieri con un livello di difficoltà elevato;
- Controlla la presenza di segnalazioni e aree critiche lungo il percorso;
- Consulta informazioni meteo aggiornate in tempo reale;
- Riceve notifiche relative a eventuali cambiamenti delle condizioni del sentiero;
- Invia segnalazioni su ostacoli o situazioni di pericolo riscontrate.

3.8.3 Persona: Paolo

Paolo è un amministratore di 40 anni che collabora con un ente locale per la gestione e la manutenzione dei sentieri. Il suo obiettivo è garantire che le informazioni disponibili sulla piattaforma siano sempre aggiornate e affidabili.

Scenario d'uso: gestione dei contenuti della piattaforma.

- Paolo accede all'area amministrativa del sistema;
- Inserisce nuovi sentieri o aggiorna quelli esistenti;
- Modera le segnalazioni inviate dagli escursionisti;
- Aggiorna lo stato dei percorsi in base alle condizioni ambientali;
- Pubblica avvisi e comunicazioni di sicurezza per gli utenti.

Attraverso queste analisi delle personas ci ha guidato le scelte progettuali di TrailSafe, contribuendo alla definizione delle funzionalità offerte dal sistema e alla progettazione di un'interfaccia intuitiva, affidabile e orientata alla sicurezza degli escursionisti.

Nella sezione seguente andremo a vedere nel dettaglio le tecnologie utilizzate all'interno di questo progetto, lo stack metodologico e le varie integrazioni usate all'interno della WebApp.

3.9 Prototipo Finale

Successivamente alla realizzazione dei mockup, è stata avviata la fase di implementazione della Web App. Durante questa fase, alcune parti inizialmente previste nei mockup hanno subito delle variazioni rispetto al progetto originale, dovute a scelte implementative e a considerazioni emerse durante lo sviluppo.

Nonostante queste differenze, la maggior parte delle funzionalità progettate nella fase di mockup è stata mantenuta anche nel prototipo finale dell'applicazione. Di seguito viene presentato il prototipo della Web App TrailSafe, illustrato attraverso immagini che mostrano le principali interfacce e funzionalità implementate.

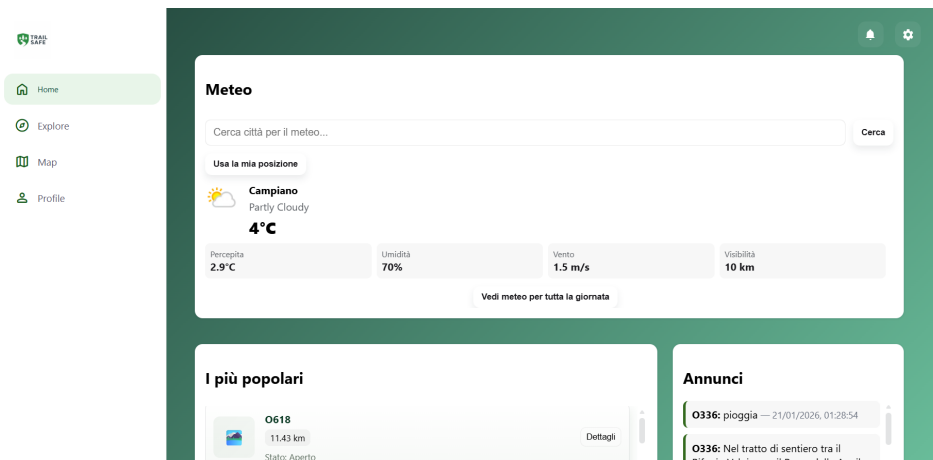


Figura 11: Prototipo finale: Home

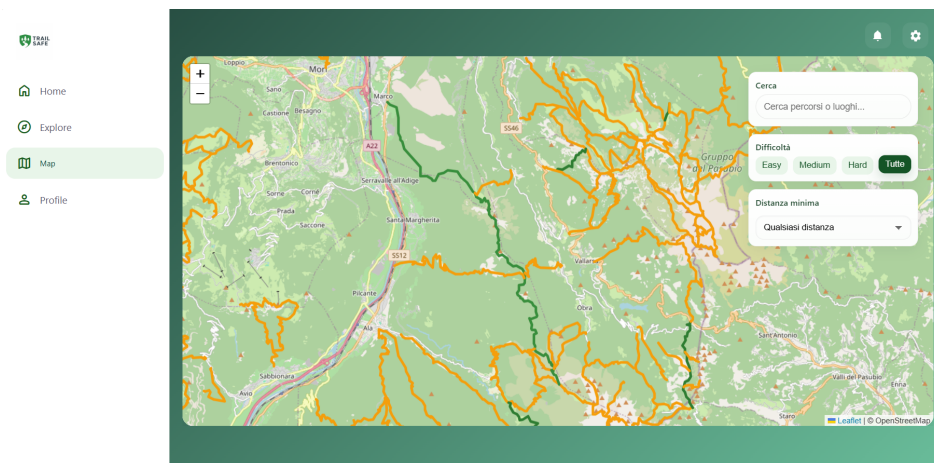


Figura 12: Prototipo finale: Map

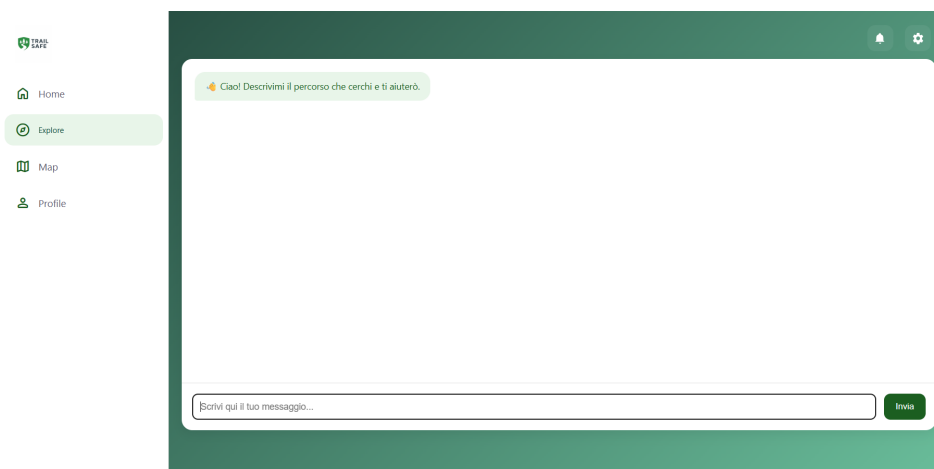


Figura 13: Prototipo finale: AI Explore

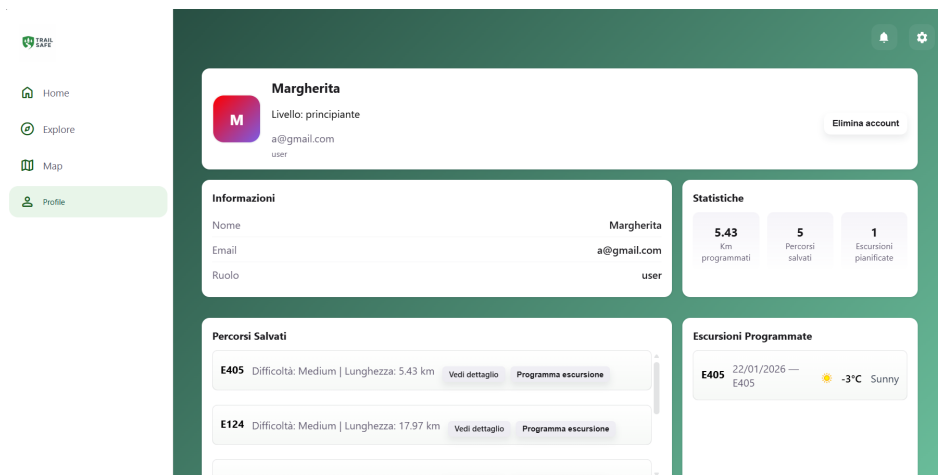


Figura 14: Prototipo finale: Profilo

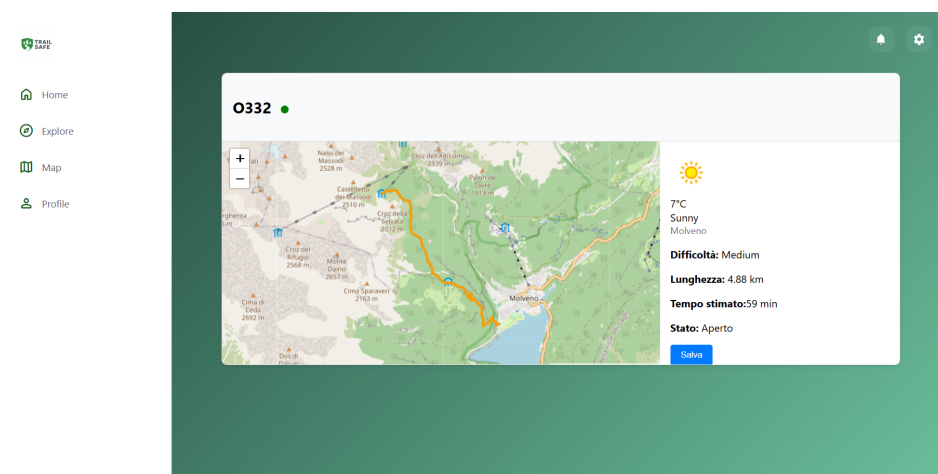


Figura 15: Prototipo finale: Dettaglio Percorso

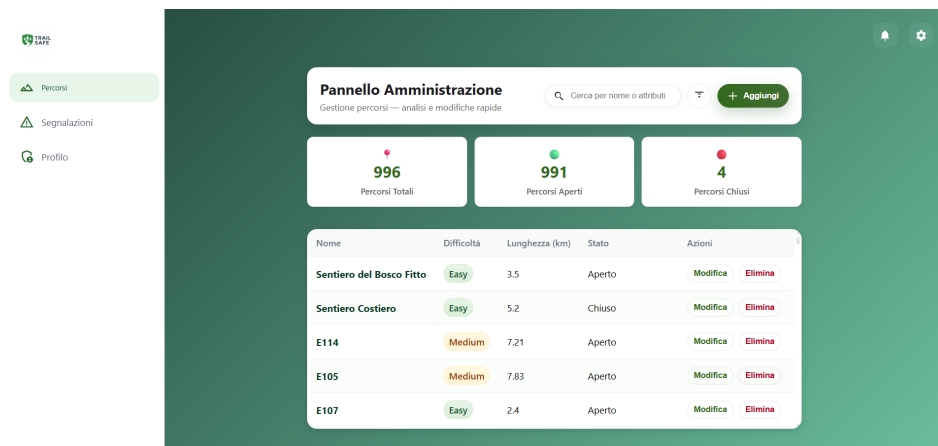


Figura 16: Prototipo finale: Admin Home page

4 Tecnologie utilizzate

Per lo sviluppo del sistema TrailSafe è stato adottato uno stack tecnologico moderno e orientato alla realizzazione di applicazioni web scalabili, modulari e facilmente manutenibili. La scelta delle tecnologie è stata guidata dalla necessità di garantire reattività dell'interfaccia, affidabilità del backend e facilità di integrazione con servizi esterni, fondamentali per fornire informazioni aggiornate in tempo reale.

L'architettura del sistema prevede una chiara separazione tra frontend e backend, consentendo un'evoluzione indipendente dei due componenti e favorendo la scalabilità dell'intera piattaforma.

4.1 Stack tecnologico MEVN

TrailSafe è sviluppato utilizzando lo stack MEVN, composto dalle seguenti tecnologie:

- **MongoDB**: database NoSQL utilizzato per la persistenza dei dati relativi agli utenti, ai sentieri, alle segnalazioni e alle notifiche. La natura documentale del database consente una gestione flessibile di strutture dati eterogenee e in continua evoluzione;
- **Express.js**: framework per Node.js utilizzato per la realizzazione delle API RESTful, che gestiscono la comunicazione tra client e server;
- **Vue.js**: framework JavaScript utilizzato per lo sviluppo del frontend, basato su un'architettura a componenti che favorisce la riusabilità e la manutenibilità del codice;
- **Node.js**: ambiente di runtime JavaScript utilizzato per l'esecuzione del backend dell'applicazione.

L'utilizzo dello stack MEVN consente di adottare JavaScript come linguaggio principale sia lato client che lato server, riducendo la complessità complessiva del progetto e facilitando la condivisione delle competenze all'interno del team di sviluppo.

4.2 Tecnologie Backend

Il backend di TrailSafe è sviluppato in Node.js ed Express.js ed è organizzato secondo un'architettura modulare. Le funzionalità del sistema sono esposte tramite API RESTful, che permettono al frontend di accedere ai dati e ai servizi offerti dalla piattaforma.

Il backend gestisce:

- l'autenticazione e l'autorizzazione degli utenti;
- la gestione dei sentieri e delle segnalazioni;
- l'integrazione con servizi esterni per l'aggiornamento delle informazioni ambientali;
- l'invio di notifiche e avvisi agli utenti.

Per la sicurezza delle comunicazioni e la protezione delle operazioni sensibili viene utilizzato un sistema di autenticazione basato su token, che consente di verificare l'identità dell'utente ad ogni richiesta al server e di limitare l'accesso alle funzionalità riservate.

4.3 Tecnologie Frontend

Il frontend dell'applicazione è sviluppato utilizzando Vue.js e segue un approccio basato su componenti. Questa struttura consente di suddividere l'interfaccia utente in elementi riutilizzabili e indipendenti, migliorando la leggibilità del codice e facilitando eventuali estensioni future.

Il frontend si occupa della visualizzazione dinamica dei dati, dell'interazione con l'utente e della gestione dello stato dell'applicazione. La comunicazione con il backend avviene tramite chiamate HTTP asincrone, garantendo un'esperienza utente fluida e reattiva.

Particolare attenzione è stata posta all'usabilità dell'interfaccia e alla compatibilità con dispositivi mobili, al fine di rendere l'applicazione accessibile anche durante l'attività escursionistica.

4.4 Servizi di mappe e informazioni ambientali

TrailSafe integra servizi esterni per la gestione delle mappe interattive e per l'aggiornamento delle informazioni ambientali e meteorologiche. Tali servizi consentono di visualizzare i sentieri su una mappa esplorabile, evidenziare punti di interesse e mostrare aree critiche lungo i percorsi.

Le informazioni ambientali vengono aggiornate dinamicamente e utilizzate dal sistema per fornire una valutazione più accurata della difficoltà e della percorribilità dei sentieri.

4.5 Integrazione del servizio di Intelligenza Artificiale

Per arricchire l'esperienza utente e fornire un supporto informativo avanzato, il sistema TrailSafe integra un servizio di Intelligenza Artificiale basato su Gemini. Tale servizio è utilizzato per rispondere alle domande degli utenti riguardanti i sentieri, le caratteristiche dei percorsi e le condizioni generali di sicurezza.

L'integrazione è stata realizzata mediante l'utilizzo di una chiave API ottenuta direttamente dal servizio Gemini. Il backend si occupa di gestire le richieste verso il servizio di Intelligenza Artificiale, inoltrando le domande degli utenti e restituendo le risposte generate al frontend.

Questa soluzione consente di:

- fornire risposte contestuali e personalizzate agli utenti;
- migliorare la comprensione delle informazioni sui sentieri;
- offrire un supporto aggiuntivo nella fase di pianificazione delle escursioni.

L'utilizzo di un servizio di Intelligenza Artificiale integrato contribuisce a rendere TrailSafe una piattaforma più interattiva e orientata al supporto dell'utente, ampliando le funzionalità offerte rispetto a una tradizionale applicazione informativa.

5 Codice

5.1 Recupero dei percorsi più popolari

L'endpoint `GET /api/trails/popular` restituisce i percorsi più popolari, calcolati in base al numero di volte in cui un percorso è stato salvato dagli utenti. Per ottenere questo risultato viene utilizzata una pipeline di aggregazione MongoDB, che analizza l'attributo `savedTrails` presente nei documenti degli utenti.

```
// GET /api/trails/popular - Most popular trails
const popularTrails = await User.aggregate([
  { $unwind: '$savedTrails' },
  { $group: { _id: '$savedTrails', count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 5 },
  { $lookup: {
    from: 'trails',
    localField: '_id',
    foreignField: '_id',
```

```

      as: 'trail'
    }},
    { $unwind: '$trail' },
    { $project: {
      _id: '$trail._id',
      name: '$trail.name',
      difficulty: '$trail.difficulty',
      length_km: '$trail.length_km',
      popularity: '$count'
    }}
  ]);

res.json(popularTrails);

```

5.2 Creazione di una segnalazione

L'endpoint POST `/api/reports` consente agli utenti autenticati di creare una nuova segnalazione relativa a un percorso. La richiesta viene validata lato server e la segnalazione viene associata all'utente che l'ha creata.

```

// Create a new report (authenticated users)
router.post('/', require('../middleware/auth'), async (req, res) => {
  try {
    const { trail, text, severity, location, imageBase64, imageUrl } = req.
      body || {};
    if (!text) return res.status(400).json({ error: 'Text is required' });

    const userId = req.user && req.user.id;
    const rep = new Report({
      trail,
      user: userId,
      text,
      severity: severity || 'low',
      location,
      imageUrl: imageUrl || imageBase64
    });

    await rep.save();
    const populated = await Report.findById(rep._id)
      .populate('trail')
      .populate('user')
      .lean();

    return res.status(201).json(populated);
  } catch (err) {
    res.status(500).json({ error: 'Server error' });
  }
});

```

5.3 Middleware di autenticazione

Il middleware di autenticazione verifica la presenza di un token JWT nell'header `Authorization`. Se il token è valido, le informazioni dell'utente vengono aggiunte alla richiesta e l'esecuzione prosegue, altrimenti viene restituito un errore di autenticazione.

```
// server/middleware/auth.js
const jwt = require('jsonwebtoken');

module.exports = function(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer '))
    return res.status(401).json({ error: 'No token provided' });

  const token = authHeader.split(' ')[1];
  try {
    const secret = process.env.JWT_SECRET || 'change-me-in-env';
    const payload = jwt.verify(token, secret);
    req.user = payload; // { id, email, role, ... }
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid token' });
  }
};
```

5.4 Registrazione di un nuovo utente

L'endpoint POST `/register` consente la creazione di un nuovo account. La password viene cifrata tramite `bcrypt` prima di essere salvata nel database. Dopo la registrazione viene generato un token JWT per l'autenticazione dell'utente.

```
// User registration
router.post('/register', async (req, res) => {
  const { name, email, password } = req.body;
  if (!email || !password)
    return res.status(400).json({ error: 'Email and password required' });

  const salt = await bcrypt.genSalt(10);
  const hash = await bcrypt.hash(password, salt);

  const user = new User({ name, email, passwordHash: hash });
  await user.save();

  const token = jwt.sign(
    { id: user._id, email: user.email, role: user.role },
    process.env.JWT_SECRET || 'change-me-in-env',
    { expiresIn: '7d' }
  );

  res.status(201).json({
    token,
    user: { id: user._id, email: user.email, name: user.name }
  });
});
```

5.5 Autenticazione utente

L'endpoint POST `/login` permette a un utente registrato di effettuare l'accesso. Le credenziali fornite vengono verificate confrontando la password con l'hash salvato nel database. In caso di successo viene restituito un token JWT.


```
// User login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });
  if (!user)
    return res.status(401).json({ error: 'Invalid_credentials' });

  const match = await bcrypt.compare(password, user.passwordHash);
  if (!match)
    return res.status(401).json({ error: 'Invalid_credentials' });

  const token = jwt.sign(
    { id: user._id, email: user.email, role: user.role },
    process.env.JWT_SECRET || 'change-me-in-env',
    { expiresIn: '7d' }
  );

  res.json({
    token,
    user: {
      id: user._id,
      email: user.email,
      name: user.name,
      role: user.role
    }
  });
});
```

5.6 Recupero delle informazioni meteo

Il componente WeatherCard recupera le informazioni meteo interrogando un endpoint backend. La richiesta può essere effettuata sia tramite il nome della città sia tramite coordinate geografiche.

```
// WeatherCard.vue (excerpt)
async loadWeather(query) {
  let url = '/api/weather';
  if (typeof query === 'string')
    url += '?city=${encodeURIComponent(query)}';
  if (query?.lat)
    url += '?lat=${query.lat}&lon=${query.lon}';

  const res = await fetch(url);
  this.weather = await res.json();
}
```

5.7 Ricerca intelligente dei percorsi

Il componente AIExplore permette all'utente di interrogare il sistema tramite linguaggio naturale. Il messaggio dell'utente viene inviato a un endpoint AI che restituisce una lista di percorsi consigliati, successivamente trasformata in una risposta leggibile.

```
// AIExplore.vue (excerpt)
async sendMessage() {
```

```

    this.messages.push({ type: 'user', text: this.userMessage });

    const res = await fetch('/api/ai/ai-search', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ query: this.userMessage })
    });

    const trails = await res.json();

    let response = 'Ecco i percorsi pi  adatti:\n';
    trails.forEach((t, i) =>
      response += `${i + 1}. "${t.name}" - Difficolt  : ${t.difficulty}, ${t.length_km} km\n`
    );

    this.messages.push({ type: 'ai', text: response });
  }

```

5.8 Filtraggio dei percorsi

Il componente Map consente di filtrare i percorsi in base a criteri come nome e difficolt . I filtri vengono trasformati in parametri di query e inviati al backend tramite una richiesta HTTP.

```

// Map.vue (excerpt)
async fetchTrails() {
  const params = new URLSearchParams();

  if (this.filters.difficulty)
    params.append('difficulty', this.filters.difficulty);
  if (this.filters.name)
    params.append('name', this.filters.name);

  const url = '/api/trails/search' +
    (params.toString() ? ('?' + params.toString()) : '');

  const res = await fetch(url);
  this.trails = await res.json();
}

computed: {
  normalizedDifficulties() {
    const set = new Set(
      (this.difficulties || []).map(x =>
        (x || '').toString().toLowerCase().trim()
      )
    );

    const out = [];
    if (set.has('easy') || set.has('facile')) out.push('Easy');
    if (set.has('medium') || set.has('intermedio')) out.push('Medium');
    if (set.has('hard') || set.has('difficile')) out.push('Hard');

    return out.length ? out : ['Easy', 'Medium', 'Hard'];
  }
}

```

5.9 Visualizzazione dei percorsi su mappa

Il componente `MapView` utilizza la libreria `Leaflet` per visualizzare i percorsi su una mappa interattiva. Ogni percorso viene rappresentato tramite una polilinea colorata in base alla difficoltà.

```
// MapView.vue (excerpt)
mounted() {
  this.map = L.map('map').setView([46.35, 11.20], 8);

  L.tileLayer(
    'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png'
  ).addTo(this.map);

  this.trailsLayer = L.layerGroup().addTo(this.map);

  if (this.trails && this.trails.length)
    this.drawTrails(this.trails);
},

methods: {
  drawTrails(trails) {
    this.trailsLayer.clearLayers();
    const bounds = L.latLngBounds([]);

    trails.forEach(trail => {
      const latlngs = trail.geometry.coordinates
        .map(([lng, lat]) => [lat, lng]);

      const color = /* map difficulty -> color */;
      const poly = L.polyline(latlngs, { color, weight: 4 })
        .bindPopup(
          '<b>${this.escapeHtml(trail.name)}</b><br>
            <button onclick="viewTrailDetail(\'${trail._id}\')">
              Vedi dettaglio
            </button>'
        );

      poly.addTo(this.trailsLayer);
      bounds.extend(latlngs);
    });

    if (bounds.isValid())
      this.map.fitBounds(bounds.pad(0.1));
  }
}
```

6 Testing

La fase di testing del sistema TrailSafe ha avuto l'obiettivo di verificare il corretto funzionamento dell'applicazione sia dal punto di vista funzionale sia dal punto di vista dell'esperienza utente. Le attività di testing sono state svolte durante l'intero ciclo di sviluppo, seguendo un approccio incrementale, al fine di individuare tempestivamente eventuali errori e anomalie.

6.1 Strategia di testing

La strategia di testing adottata prevede la verifica delle singole componenti del sistema e delle funzionalità offerte all'interno delle diverse pagine dell'applicazione. In particolare, sono state effettuate attività di:

- testing delle funzionalità del backend;
- testing delle interazioni tra frontend e backend;
- testing delle pagine dell'applicazione e delle funzionalità in esse presenti.

Questo approccio ha consentito di garantire la coerenza del comportamento del sistema nelle diverse situazioni di utilizzo.

6.2 Testing del Backend

Il backend è stato testato verificando il corretto funzionamento delle API RESTful e delle principali operazioni del sistema. In particolare, sono stati testati:

- i meccanismi di autenticazione e autorizzazione;
- la gestione degli errori e delle risposte del server;
- l'integrazione con servizi esterni, come il servizio di Intelligenza Artificiale basato su Gemini.

Il testing del backend ha permesso di garantire la stabilità delle API e la corretta gestione delle richieste provenienti dal frontend.

6.3 Testing del Frontend e delle pagine

Il frontend è stato testato verificando il corretto funzionamento delle diverse pagine dell'applicazione e delle funzionalità disponibili per l'utente. In particolare, sono state testate:

- la pagina di autenticazione, verificando la correttezza delle operazioni di login e registrazione;
- la pagina della mappa interattiva, controllando la visualizzazione dei sentieri, dei punti di interesse e delle aree critiche;
- le pagine dedicate alla gestione dei percorsi salvati;
- le pagine amministrative, verificando le operazioni di creazione, modifica e rimozione dei sentieri;
- le funzionalità di invio e visualizzazione delle segnalazioni.

Per ciascuna pagina è stato verificato il corretto comportamento delle funzionalità disponibili, assicurando che le azioni dell'utente producessero i risultati attesi e che le informazioni visualizzate fossero coerenti con i dati restituiti dal backend.

6.4 Testing delle funzionalità principali

Particolare attenzione è stata posta al testing delle funzionalità critiche del sistema, fondamentali per garantire la sicurezza degli escursionisti. Tra queste:

- aggiornamento dinamico delle informazioni ambientali;
- ricezione e visualizzazione delle notifiche;
- interazione con il servizio di Intelligenza Artificiale per la consultazione di informazioni sui sentieri.

Le funzionalità sono state testate simulando scenari di utilizzo realistici, al fine di verificare il comportamento del sistema in condizioni operative reali.

Le attività di testing hanno evidenziato il corretto funzionamento delle principali pagine e funzionalità del sistema TrailSafe. Gli eventuali problemi riscontrati sono stati corretti durante le fasi di sviluppo, contribuendo a migliorare l'affidabilità, la stabilità e l'usabilità complessiva dell'applicazione prima della fase di deployment.

7 Deployment

La fase di deployment del sistema TrailSafe è stata progettata per consentire un avvio rapido dell'ambiente di sviluppo e una chiara separazione tra il backend e il frontend dell'applicazione. Durante lo sviluppo, i due componenti vengono eseguiti come servizi indipendenti su porte differenti, facilitando il testing e il debug del sistema.

7.1 Setup iniziale e scaffolding

Per agevolare l'avvio del progetto è stato predisposto uno scaffolding minimale, che consente di iniziare rapidamente lo sviluppo delle funzionalità principali. La struttura del progetto è suddivisa come segue:

- **server/**: contiene il backend dell'applicazione, sviluppato con Node.js ed Express.js;
- **client/**: contiene il frontend dell'applicazione, sviluppato con Vue 3 e Vite.

7.2 Avvio dei servizi in ambiente di sviluppo

In ambiente di sviluppo, il backend e il frontend vengono avviati separatamente ed eseguiti su porte differenti.

Il backend viene avviato sulla porta 3000 ed è accessibile tramite l'indirizzo:

`http://localhost:3000`

Da questo indirizzo è possibile accedere alle API REST del sistema, come ad esempio: `GET /api/status` e `GET /api/trails`.

Il frontend viene avviato sulla porta 5173 ed è accessibile tramite l'indirizzo:

`http://localhost:5173`

Da questo indirizzo è possibile accedere all'interfaccia grafica dell'applicazione e interagire con tutte le funzionalità offerte dal sistema TrailSafe.

7.3 Comandi di avvio

L'avvio dei servizi avviene tramite i seguenti comandi:

Backend:

```
cd server
npm install
npm run dev
```

Frontend:

```
cd client
npm install
npm run dev
```

Questa configurazione consente di eseguire simultaneamente client e server in due terminali distinti.

7.4 Configurazione del proxy

Il client frontend, basato su Vite, è configurato per proxyare tutte le richieste dirette al percorso `/api` verso il backend in esecuzione su `http://localhost:3000`. In questo modo, le chiamate API effettuate dal frontend vengono inoltrate correttamente al server senza necessità di varie configurazioni aggiuntive.

7.5 Gestione dell'utente amministratore e cliente

Per l'accesso alle funzionalità amministrative del sistema sono state definite credenziali iniziali per l'utente amministratore, configurate tramite variabili d'ambiente:

- `ADMIN_USERNAME=admin`
- `ADMIN_PASSWORD=adminpass`
- `ADMIN_EMAIL=admin@trailsafe.local`

Queste credenziali permettono l'accesso all'area amministrativa della piattaforma e la gestione dei contenuti del sistema.

Invece per quanto riguarda la parte del utente basta iscriversi con un username ed una password alla WebApp in maniera semplice, dopo essersi registrati l'amministratore vedrà l'utente iscritto e potrà gestire l'entità del singolo utente.

8 Conclusioni e sviluppi futuri

Il progetto TrailSafe è stato un'esperienza particolarmente interessante e formativa, che ha permesso a noi del gruppo di lavoro di affrontare in modo pratico le principali fasi di sviluppo di un'applicazione web moderna. Il progetto è nato dall'analisi dei bisogni degli utenti, in particolare degli escursionisti, con l'obiettivo di realizzare uno strumento semplice da utilizzare ma allo stesso tempo utile e affidabile per migliorare la sicurezza durante le attività outdoor.

Durante lo sviluppo di TrailSafe è stato possibile apprendere e approfondire l'utilizzo di nuove tecnologie e framework per la creazione di applicazioni web. In particolare, l'adozione dello stack MEVN, l'integrazione di servizi esterni come mappe, notifiche e un servizio di Intelligenza Artificiale, ha consentito di comprendere come realizzare applicazioni modulari e facilmente estendibili, anche lavorando in un gruppo di dimensioni ridotte.

Un aspetto rilevante del progetto è stato l'approccio orientato all'utente. Partire dai bisogni reali degli utenti ha guidato le scelte progettuali e di implementazione, rendendo l'applicazione più intuitiva e accessibile. Questo ha evidenziato come lo sviluppo di un buon prodotto software non dipenda esclusivamente dalle tecnologie utilizzate, ma anche dalla capacità di progettare soluzioni che rispondano in modo concreto alle esigenze dell'utente finale.

Dal punto di vista organizzativo, il lavoro di gruppo ha permesso di migliorare la collaborazione e la suddivisione dei compiti, favorendo il confronto sulle scelte tecniche e progettuali. Lavorare in due ha reso possibile affrontare le difficoltà in modo più efficace e acquisire una maggiore consapevolezza del processo di sviluppo software.

Per quanto riguarda gli sviluppi futuri, TrailSafe potrebbe essere ulteriormente ampliato introducendo nuove funzionalità, come una versione mobile dedicata così da usare l'app anche all'esterno per essere aggiornati in tempo reale sulle condizioni meteo e dei percorsi, l'integrazione con dispositivi o sensori ambientali e un utilizzo più avanzato dell'Intelligenza Artificiale (magari aggiungendo anche dei costi di servizio) per fornire suggerimenti personalizzati e previsioni sui rischi dei percorsi.

Il progetto TrailSafe rappresenta un'esperienza completa che ci ha permesso di applicare conoscenze teoriche in un contesto pratico, imparare nuove tecnologie e sviluppare un'applicazione web in modo efficace, partendo dai bisogni degli utenti e lavorando in collaborazione all'interno di un piccolo gruppo come il nostro.